JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class ArrayDeque<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.ArrayDeque<E>

Type Parameters:

`E - the type of elements held in this collection`

All Implemented Interfaces:

`Serializable`, `Cloneable`, `Iterable<E>`, `Collection<E>`, `Deque<E>`, `Queue<E>`

```
public class ArrayDeque<E>
extends AbstractCollection<E>
implements Deque<E>, Cloneable, Serializable
```

Resizable-array implementation of the `Deque` interface. Array deques have no capacity restrictions; they grow as necessary to support usage. They are not thread-safe; in the absence of external synchronization, they do not support concurrent access by multiple threads. Null elements are prohibited. This class is likely to be faster than `Stack` when used as a stack, and faster than `LinkedList` when used as a queue.

Most `ArrayDeque` operations run in amortized constant time. Exceptions include `remove`, `removeFirstOccurrence`, `removeLastOccurrence`, `contains`, `iterator.remove()`, and the bulk operations, all of which run in linear time.

The iterators returned by this class's `iterator` method are *fail-fast*: If the deque is modified at any time after the iterator is created, in any way except through the iterator's own `remove` method, the iterator will generally throw a `ConcurrentModificationException`. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future.

Note that the fail-fast behavior of an iterator cannot be guaranteed as it is, generally speaking, impossible to make any hard guarantees in the presence of unsynchronized concurrent modification. Fail-fast iterators throw `ConcurrentModificationException` on a best-effort basis. Therefore, it would be wrong to write a program that depended on this exception for its correctness: *the fail-fast behavior of iterators should be used only to detect bugs.*

This class and its iterator implement all of the *optional* methods of the `Collection` and `Iterator` interfaces.

This class is a member of the Java Collections Framework.

Since:

1.6

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`ArrayDeque()`	Constructs an empty array deque with an initial capacity sufficient to hold 16 elements.
`ArrayDeque(Collection<? extends E> c)`	Constructs a deque containing the elements of the specified collection, in the order they are returned by the collection's iterator.
`ArrayDeque(int numElements)`	Constructs an empty array deque with an initial capacity sufficient to hold the specified number of elements.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
boolean	`add(E e)`	Inserts the specified element

		at the end of this deque.
void	**addFirst**(**E** e)	Inserts the specified element at the front of this deque.
void	**addLast**(**E** e)	Inserts the specified element at the end of this deque.
void	**clear**()	Removes all of the elements from this deque.
ArrayDeque<**E**>	**clone**()	Returns a copy of this deque.
boolean	**contains**(**Object** o)	Returns `true` if this deque contains the specified element.
Iterator<**E**>	**descendingIterator**()	Returns an iterator over the elements in this deque in reverse sequential order.
E	**element**()	Retrieves, but does not remove, the head of the queue represented by this deque.
E	**getFirst**()	Retrieves, but does not remove, the first element of this deque.
E	**getLast**()	Retrieves, but does not remove, the last element of this deque.
boolean	**isEmpty**()	Returns `true` if this deque

		contains no elements.
`Iterator<E>`	`iterator()`	Returns an iterator over the elements in this deque.
boolean	`offer(E e)`	Inserts the specified element at the end of this deque.
boolean	`offerFirst(E e)`	Inserts the specified element at the front of this deque.
boolean	`offerLast(E e)`	Inserts the specified element at the end of this deque.
`E`	`peek()`	Retrieves, but does not remove, the head of the queue represented by this deque, or returns `null` if this deque is empty.
`E`	`peekFirst()`	Retrieves, but does not remove, the first element of this deque, or returns `null` if this deque is empty.
`E`	`peekLast()`	Retrieves, but does not remove, the last element of this deque, or returns `null` if this deque is empty.
`E`	`poll()`	

		Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.
E	**pollFirst**()	Retrieves and removes the first element of this deque, or returns `null` if this deque is empty.
E	**pollLast**()	Retrieves and removes the last element of this deque, or returns `null` if this deque is empty.
E	**pop**()	Pops an element from the stack represented by this deque.
void	**push**(**E** e)	Pushes an element onto the stack represented by this deque.
E	**remove**()	Retrieves and removes the head of the queue represented by this deque.
boolean	**remove**(**Object** o)	Removes a single instance of the specified element from this deque.
E	**removeFirst**()	Retrieves and removes the first

		element of this deque.
boolean	**removeFirstOccurrence**(**Object** o)	Removes the first occurrence of the specified element in this deque (when traversing the deque from head to tail).
E	**removeLast**()	Retrieves and removes the last element of this deque.
boolean	**removeLastOccurrence**(**Object** o)	Removes the last occurrence of the specified element in this deque (when traversing the deque from head to tail).
int	**size**()	Returns the number of elements in this deque.
Spliterator<**E**>	**spliterator**()	Creates a ***late-binding*** and *fail-fast* **Spliterator** over the elements in this deque.
Object[]	**toArray**()	Returns an array containing all of the elements in this deque in proper sequence (from first to last element).
<T> T[]	**toArray**(T[] a)	Returns an array containing all of the elements in this deque in proper sequence (from first to last element); the

		runtime type of the returned array is that of the specified array.

Methods inherited from class java.util.AbstractCollection

addAll, containsAll, removeAll, retainAll, toString

Methods inherited from class java.lang.Object

equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.Collection

addAll, containsAll, equals, hashCode, parallelStream, removeAll, removeIf, retainAll, stream

Methods inherited from interface java.lang.Iterable

forEach

Constructor Detail

ArrayDeque

```
public ArrayDeque()
```

Constructs an empty array deque with an initial capacity sufficient to hold 16 elements.

ArrayDeque

```
public ArrayDeque(int numElements)
```

Constructs an empty array deque with an initial capacity sufficient to hold the specified number of elements.

Parameters:

`numElements - lower bound on initial capacity of the deque`

ArrayDeque

```
public ArrayDeque(Collection<? extends E> c)
```

Constructs a deque containing the elements of the specified collection, in the order they are returned by the collection's iterator. (The first element returned by the collection's iterator becomes the first element, or *front* of the deque.)

Parameters:

`c - the collection whose elements are to be placed into the deque`

Throws:

`NullPointerException - if the specified collection is null`

Method Detail

addFirst

```
public void addFirst(E e)
```

Inserts the specified element at the front of this deque.

Specified by:

`addFirst in interface Deque<E>`

Parameters:

`e - the element to add`

Throws:

`NullPointerException - if the specified element is null`

addLast

```
public void addLast(E e)
```

Inserts the specified element at the end of this deque.

This method is equivalent to `add(E)`.

Specified by:

`addLast in interface Deque<E>`

Parameters:

`e - the element to add`

Throws:

NullPointerException - if the specified element is null

offerFirst

public boolean offerFirst(E e)

Inserts the specified element at the front of this deque.

Specified by:

offerFirst in interface Deque<E>

Parameters:

e - the element to add

Returns:

true (as specified by Deque.offerFirst(E))

Throws:

NullPointerException - if the specified element is null

offerLast

public boolean offerLast(E e)

Inserts the specified element at the end of this deque.

Specified by:

offerLast in interface Deque<E>

Parameters:

e - the element to add

Returns:

true (as specified by Deque.offerLast(E))

Throws:

NullPointerException - if the specified element is null

removeFirst

public E removeFirst()

Description copied from interface: Deque

Retrieves and removes the first element of this deque. This method differs from pollFirst only in that it throws an exception if this deque is empty.

Specified by:

removeFirst in interface Deque<E>

Returns:

the head of this deque

Throws:

NoSuchElementException - if this deque is empty

removeLast

public E removeLast()

Description copied from interface: Deque

Retrieves and removes the last element of this deque. This method differs from pollLast only in that it throws an exception if this deque is empty.

Specified by:

removeLast in interface Deque<E>

Returns:

the tail of this deque

Throws:

NoSuchElementException - if this deque is empty

pollFirst

public E pollFirst()

Description copied from interface: Deque

Retrieves and removes the first element of this deque, or returns null if this deque is empty.

Specified by:

pollFirst in interface Deque<E>

Returns:

the head of this deque, or null if this deque is empty

pollLast

public E pollLast()

Description copied from interface: Deque

Retrieves and removes the last element of this deque, or returns null if this deque is empty.

Specified by:

pollLast in interface Deque<E>

Returns:

the tail of this deque, or null if this deque is empty

getFirst

```
public E getFirst()
```

Description copied from interface: Deque

Retrieves, but does not remove, the first element of this deque. This method differs from peekFirst only in that it throws an exception if this deque is empty.

Specified by:

getFirst in interface Deque<E>

Returns:

the head of this deque

Throws:

NoSuchElementException - if this deque is empty

getLast

```
public E getLast()
```

Description copied from interface: Deque

Retrieves, but does not remove, the last element of this deque. This method differs from peekLast only in that it throws an exception if this deque is empty.

Specified by:

getLast in interface Deque<E>

Returns:

the tail of this deque

Throws:

NoSuchElementException - if this deque is empty

peekFirst

```
public E peekFirst()
```

Description copied from interface: Deque

Retrieves, but does not remove, the first element of this deque, or returns null if this deque is empty.

Specified by:

peekFirst in interface Deque<E>

Returns:

the head of this deque, or null if this deque is empty

peekLast

public E peekLast()

Description copied from interface: Deque

Retrieves, but does not remove, the last element of this deque, or returns null if this deque is empty.

Specified by:

peekLast in interface Deque<E>

Returns:

the tail of this deque, or null if this deque is empty

removeFirstOccurrence

public boolean removeFirstOccurrence(Object o)

Removes the first occurrence of the specified element in this deque (when traversing the deque from head to tail). If the deque does not contain the element, it is unchanged. More formally, removes the first element e such that o.equals(e) (if such an element exists). Returns true if this deque contained the specified element (or equivalently, if this deque changed as a result of the call).

Specified by:

removeFirstOccurrence in interface Deque<E>

Parameters:

o - element to be removed from this deque, if present

Returns:

true if the deque contained the specified element

removeLastOccurrence

public boolean removeLastOccurrence(Object o)

Removes the last occurrence of the specified element in this deque (when traversing the deque from head to tail). If the deque does not contain the element, it is unchanged. More formally, removes the last element e such that o.equals(e) (if such an element exists). Returns true if this deque

contained the specified element (or equivalently, if this deque changed as a result of the call).

Specified by:

`removeLastOccurrence` in interface `Deque<E>`

Parameters:

`o - element to be removed from this deque, if present`

Returns:

`true if the deque contained the specified element`

add

`public boolean add(E e)`

Inserts the specified element at the end of this deque.

This method is equivalent to `addLast(E)`.

Specified by:

`add` in interface `Collection<E>`

Specified by:

`add` in interface `Deque<E>`

Specified by:

`add` in interface `Queue<E>`

Overrides:

`add` in class `AbstractCollection<E>`

Parameters:

`e - the element to add`

Returns:

`true (as specified by Collection.add(E))`

Throws:

`NullPointerException - if the specified element is null`

offer

`public boolean offer(E e)`

Inserts the specified element at the end of this deque.

This method is equivalent to `offerLast(E)`.

Specified by:

`offer` in interface `Deque<E>`

Specified by:

`offer` in interface `Queue<E>`

Parameters:

`e` - the element to add

Returns:

`true` (as specified by `Queue.offer(E)`)

Throws:

`NullPointerException` - if the specified element is null

remove

`public E remove()`

Retrieves and removes the head of the queue represented by this deque. This method differs from `poll` only in that it throws an exception if this deque is empty.

This method is equivalent to `removeFirst()`.

Specified by:

`remove` in interface `Deque<E>`

Specified by:

`remove` in interface `Queue<E>`

Returns:

the head of the queue represented by this deque

Throws:

`NoSuchElementException` - if this deque is empty

poll

`public E poll()`

Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.

This method is equivalent to `pollFirst()`.

Specified by:

`poll` in interface `Deque<E>`

Specified by:

`poll` in interface `Queue<E>`

Returns:

the head of the queue represented by this deque, or null if this deque is empty

element

```
public E element()
```

Retrieves, but does not remove, the head of the queue represented by this deque. This method differs from peek only in that it throws an exception if this deque is empty.

This method is equivalent to getFirst().

Specified by:

element in interface Deque<E>

Specified by:

element in interface Queue<E>

Returns:

the head of the queue represented by this deque

Throws:

NoSuchElementException - if this deque is empty

peek

```
public E peek()
```

Retrieves, but does not remove, the head of the queue represented by this deque, or returns null if this deque is empty.

This method is equivalent to peekFirst().

Specified by:

peek in interface Deque<E>

Specified by:

peek in interface Queue<E>

Returns:

the head of the queue represented by this deque, or null if this deque is empty

push

```
public void push(E e)
```

Pushes an element onto the stack represented by this deque. In other words, inserts the element at the front of this deque.

This method is equivalent to `addFirst(E)`.

Specified by:

`push` in interface `Deque<E>`

Parameters:

`e` - the element to push

Throws:

`NullPointerException` - if the specified element is null

pop

```
public E pop()
```

Pops an element from the stack represented by this deque. In other words, removes and returns the first element of this deque.

This method is equivalent to `removeFirst()`.

Specified by:

`pop` in interface `Deque<E>`

Returns:

the element at the front of this deque (which is the top of the stack represented by this deque)

Throws:

`NoSuchElementException` - if this deque is empty

size

```
public int size()
```

Returns the number of elements in this deque.

Specified by:

`size` in interface `Collection<E>`

Specified by:

`size` in interface `Deque<E>`

Specified by:

`size` in class `AbstractCollection<E>`

Returns:

the number of elements in this deque

isEmpty

```
public boolean isEmpty()
```

Returns `true` if this deque contains no elements.

Specified by:

`isEmpty` in interface `Collection<E>`

Overrides:

`isEmpty` in class `AbstractCollection<E>`

Returns:

`true` if this deque contains no elements

iterator

```
public Iterator<E> iterator()
```

Returns an iterator over the elements in this deque. The elements will be ordered from first (head) to last (tail). This is the same order that elements would be dequeued (via successive calls to `remove()` or popped (via successive calls to `pop()`).

Specified by:

`iterator` in interface `Iterable<E>`

Specified by:

`iterator` in interface `Collection<E>`

Specified by:

`iterator` in interface `Deque<E>`

Specified by:

`iterator` in class `AbstractCollection<E>`

Returns:

an iterator over the elements in this deque

descendingIterator

```
public Iterator<E> descendingIterator()
```

Description copied from interface: `Deque`

Returns an iterator over the elements in this deque in reverse sequential order. The elements will be returned in order from last (tail) to first (head).

Specified by:

`descendingIterator` in interface `Deque<E>`

Returns:

an iterator over the elements in this deque in reverse sequence

contains

```
public boolean contains(Object o)
```

Returns `true` if this deque contains the specified element. More formally, returns `true` if and only if this deque contains at least one element `e` such that `o.equals(e)`.

Specified by:

`contains` in interface `Collection<E>`

Specified by:

`contains` in interface `Deque<E>`

Overrides:

`contains` in class `AbstractCollection<E>`

Parameters:

`o` - object to be checked for containment in this deque

Returns:

`true` if this deque contains the specified element

remove

```
public boolean remove(Object o)
```

Removes a single instance of the specified element from this deque. If the deque does not contain the element, it is unchanged. More formally, removes the first element `e` such that `o.equals(e)` (if such an element exists). Returns `true` if this deque contained the specified element (or equivalently, if this deque changed as a result of the call).

This method is equivalent to `removeFirstOccurrence(Object)`.

Specified by:

`remove` in interface `Collection<E>`

Specified by:

`remove` in interface `Deque<E>`

Overrides:

`remove` in class `AbstractCollection<E>`

Parameters:

`o` - element to be removed from this deque, if present

Returns:

`true` if this deque contained the specified element

clear

```
public void clear()
```

Removes all of the elements from this deque. The deque will be empty after this call returns.

Specified by:

`clear` in interface `Collection<E>`

Overrides:

`clear` in class `AbstractCollection<E>`

toArray

```
public Object[] toArray()
```

Returns an array containing all of the elements in this deque in proper sequence (from first to last element).

The returned array will be "safe" in that no references to it are maintained by this deque. (In other words, this method must allocate a new array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Returns:

an array containing all of the elements in this deque

toArray

```
public <T> T[] toArray(T[] a)
```

Returns an array containing all of the elements in this deque in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array. If the deque fits in the specified array, it is returned therein. Otherwise, a new array is allocated with the runtime type of the specified array and the size of this deque.

If this deque fits in the specified array with room to spare (i.e., the array has more elements than this deque), the element in the array immediately following the end of the deque is set to `null`.

Like the `toArray()` method, this method acts as bridge between array-based and collection-based APIs. Further, this method allows precise control over the runtime type of the output array, and may, under certain circumstances, be used to save allocation costs.

Suppose `x` is a deque known to contain only strings. The following code can be used to dump the deque into a newly allocated array of `String`:

```
String[] y = x.toArray(new String[0]);
```

Note that `toArray(new Object[0])` is identical in function to `toArray()`.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Type Parameters:

`T - the runtime type of the array to contain the collection`

Parameters:

`a - the array into which the elements of the deque are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose`

Returns:

`an array containing all of the elements in this deque`

Throws:

`ArrayStoreException - if the runtime type of the specified array is not a supertype of the runtime type of every element in this deque`

`NullPointerException - if the specified array is null`

clone

`public ArrayDeque<E> clone()`

Returns a copy of this deque.

Overrides:

`clone` in class `Object`

Returns:

`a copy of this deque`

See Also:

Cloneable

spliterator

```
public Spliterator<E> spliterator()
```

Creates a *late-binding* and *fail-fast* Spliterator over the elements in this deque.

The Spliterator reports Spliterator.SIZED, Spliterator.SUBSIZED, Spliterator.ORDERED, and Spliterator.NONNULL. Overriding implementations should document the reporting of additional characteristic values.

Specified by:

spliterator in interface Iterable<E>

Specified by:

spliterator in interface Collection<E>

Returns:

a Spliterator over the elements in this deque

Since:

1.8

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.